

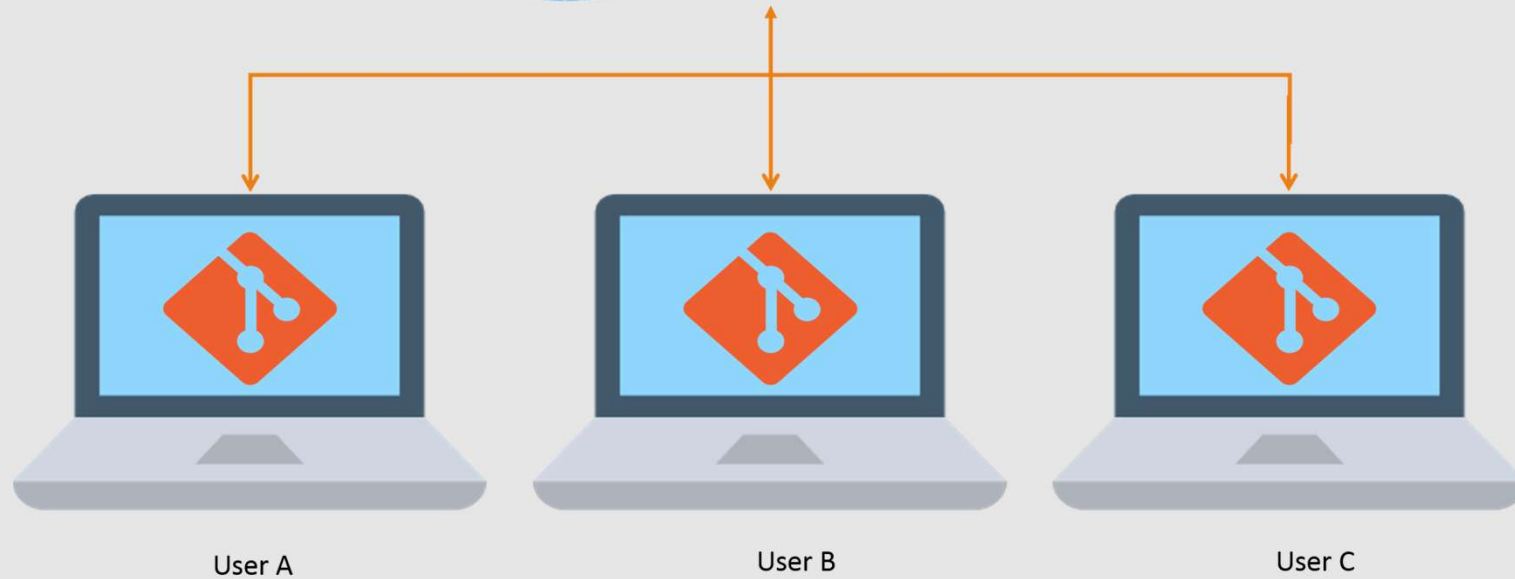
Extra: Git y GitHub

Sistemas de Control de Versiones Distribuido

GitHub: Gestor de proyectos

01

Git y GitHub



PROMETEO

01 Git



¿Por qué Git?

- **Git es un sistema libre.** No puede haber limitaciones contractuales sobre el uso de Git. Sea cual sea el número de colaboradores, de proyectos o actualizaciones, Git siempre será gratuito.
- Git posee también todos los beneficios derivados de los **DVCS**. Es decir, que es posible trabajar en modo desconectado con su repositorio manteniendo las características avanzadas de colaboración cuando se está conectado.
- Git tiene un **rendimiento excelente**. Git incorpora muchas herramientas internas para mejorar su rendimiento (en particular para el uso de las ramas).
- Git es actualmente el sistema más **popular** y gana en popularidad cada año. Es difícil contar con estadísticas precisas sobre los sistemas de gestión de versiones instalados y usados en las empresas.

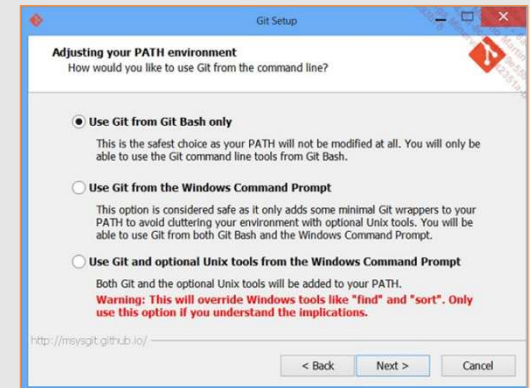
01 Git

Instalación en Linux

- Para instalar Git en un sistema Debian a partir de paquetes preexistentes, se puede utilizar el gestor de paquetes apt-get. Después de haber abierto el CLI, debemos utilizar el siguiente comando para instalar Git: `sudo apt-get install git`
- Una vez que la instalación ha terminado, para probar la instalación y obtener la versión de Git que fue instalada, debemos utilizar el siguiente comando: `git --version`.

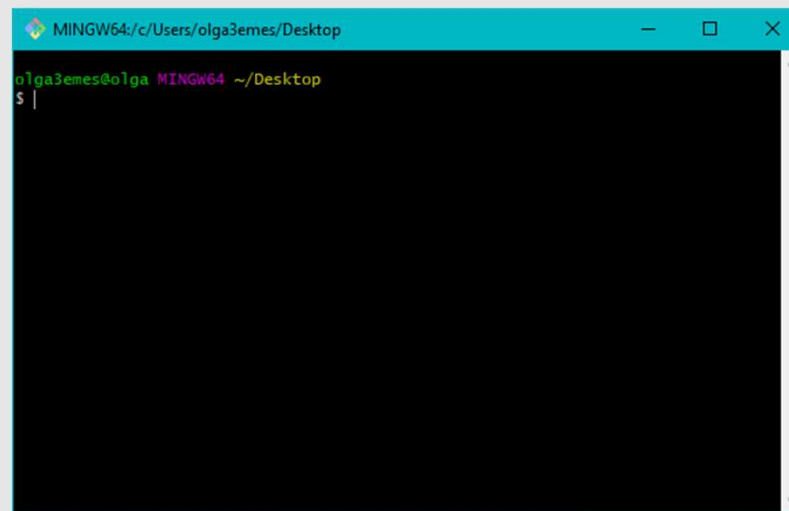
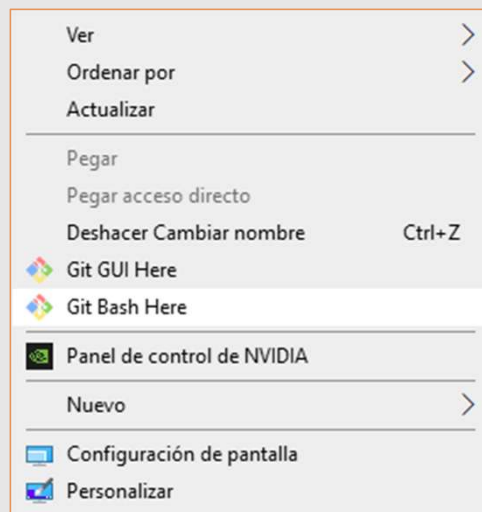
Instalación en Mac y Windows

- Para descargar Git debemos ir al sitio oficial, en la dirección : <https://git-scm.com/>
- A partir de la información enviada por el navegador, detecta la versión que se ha de instalar y ofrece un enlace de descarga en la parte inferior derecha en una imagen de Mac o Windows. Hay que descargar la aplicación. Y seguir el instalador.
- Con Mac también podemos usar brew .



01 Git

Una vez instalado Git, en nuestro sistema Windows aparecerá algo parecido a:



Mientras que en Linux y Mac no tiene por qué aparecer, dado que sus terminales ya soportan los comandos necesarios del kernel de Linux.

01

Git y GitHub

¿Qué es Github?

Plataforma para la gestión de repositorios Git y desarrollo colaborativo.

Para hacer uso de Github, nos basta con registrarnos en:
<https://github.com/>

Al crear una cuenta debemos verificar nuestro correo electrónico.

Se recomienda encarecidamente el uso de una dirección de correo electrónico profesional.

<https://learn.microsoft.com/es-es/shows/visual-studio-code/using-git-with-visual-studio-code-official-beginner-tutorial>



Comandos básicos

Después de haber instalado Git, es muy útil saber cómo acceder a su ayuda. Puede ser necesario obtener más información sobre un comando y para ello necesitaremos la ayuda. Para mostrar la ayuda general que lista los comandos principales de Git, hay que utilizar el siguiente comando:

```
git --help
```

Este comando permite tener una lista y una breve explicación (en inglés) de los comandos principales de Git.

Para obtener una ayuda más completa sobre un comando específico de Git, hay que utilizar la siguiente sintaxis:

```
git comando --help
```

Git propone una serie de comandos que se dividen en dos categorías:

- Los comandos de porcelana (porcelain command): Estos son los comandos de alto nivel. Son los comandos que se utilizan a diario y para los que Git propone una serie de controles. Ej.: git commit o git add.
- Los comandos de fontanería (plumbing commands): Estos son los comandos de bajo nivel, es decir, que manipulan directamente la información que constituye el núcleo de Git. Estos comandos pueden ser destructivos para el repositorio y suelen ser aplicados por usuarios que dominan Git.

Para ver todos hay que usar:

```
git help -a
```

Comandos básicos

Configuración requerida

Existen dos parámetros de configuración que son necesarios antes de cualquier uso de Git. Estos parámetros de configuración son la dirección de e-mail del usuario del repositorio y su nombre.

1. Configurar el nombre de usuario

```
git config --global user.name "Nombre Apellido"
```

2. Configurar el e-mail de usuario

```
git config --global user.email email@dominio.com
```

Ya estamos listos para inicializar un repositorio local, usaremos:

```
git init
```

¡AVISO! Si tu ordenador está compartido con otros compañeros es muy posible que si usas Windows, necesites dirigirte al Administrador de credenciales y borrar los datos de aquellos usuarios que no estén en este momento utilizando el PC.

Comandos básicos

```
MINGW64:/c/Users/olga3emes/Desktop/repositorio/.git

olga3emes@olga MINGW64 ~/Desktop
$ mkdir repositorio

olga3emes@olga MINGW64 ~/Desktop
$ cd repositorio

olga3emes@olga MINGW64 ~/Desktop/repositorio
$ git init
Initialized empty Git repository in C:/Users/olga3emes/Desktop/repositorio/.git/

olga3emes@olga MINGW64 ~/Desktop/repositorio (master)
$ ls -la
total 36
drwxr-xr-x 1 olga3emes 197121 0 jul.  1 20:21 ./
drwxr-xr-x 1 olga3emes 197121 0 jul.  1 20:21 ../
drwxr-xr-x 1 olga3emes 197121 0 jul.  1 20:21 .git/

olga3emes@olga MINGW64 ~/Desktop/repositorio (master)
$ cd .git

olga3emes@olga MINGW64 ~/Desktop/repositorio/.git (GIT_DIR!)
$ ls
config  description  HEAD  hooks/  info/  objects/  refs/

olga3emes@olga MINGW64 ~/Desktop/repositorio/.git (GIT_DIR!)
$ |
```

Comandos básicos

Concepto básico: "Las 3 zonas de un repositorio Git"

Working Copy | Staging Area | Repository

working copy: la carpeta de nuestro proyecto

staging area: donde pondremos los archivos a "trackear" (también se le conoce por index o caché)

repository: donde se almacena toda la información de los cambios

Comparar entre las 3 zonas:

git diff (Working Copy → Staging Area)
git diff --staged (Staging Area → Repositorio)
git diff HEAD (Working Copy → Repositorio)



Extensión para ver los cambios:

<https://marketplace.visualstudio.com/items?itemName=mhutchie.git-graph>

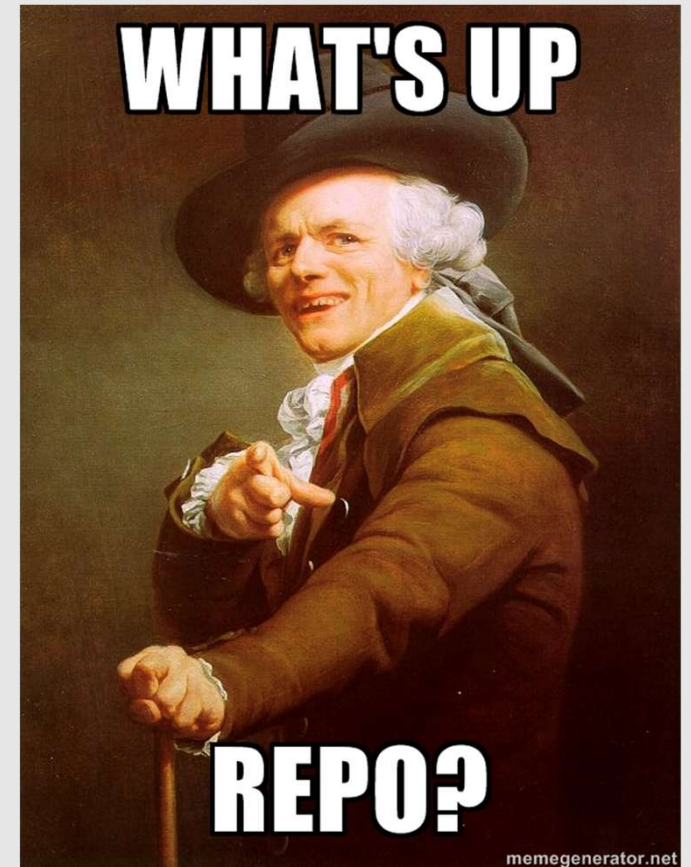
comandos básicos

¿CÓMO ESTÁ MI REPOSITORIO?

git status

Nos resume el estado de nuestro repositorio:

- qué archivos están en el staging area y cuales no
- qué archivos son "trackeados"
- qué archivos han sido modificados



Comandos básicos

DE WORKING COPY A STAGING AREA

`git add`

Añade el archivo al staging area

`git add <filename>`

Añade el directorio al staging area

`git add <folder>`

Añade los archivos .md al staging area

`git add *.md`

¿Y si me equivoco?

`git reset HEAD o <filename> o <folder> o *.md`

`git checkout --<filename>`

Para que sea más fácil:

Añadimos al staging area todos los cambios.

`git add -A / --all`

Añadimos los cambios y nuevos archivos, pero no los eliminados.

`git add .`

Añadimos los archivos modificados y eliminados, pero no los nuevos.

`git add -u`

Comandos básicos

DEL STAGING AREA AL REPOSITORIO:

Hacemos un commit

¿Qué es un commit? Hacer un commit es como “hacer una foto”

Vale..., pero ¿qué es un commit?

- Un commit es un paquete que contiene:
- Uno o más «hunks» (el resultado de un diff)
- Un mensaje que describe qué cambios van en este commit
- Un hash SHA para identificar el commit (ID)
- Un enlace con su "commit padre"

`git commit`



Nos abrirá nuestro editor de texto para escribir un mensaje descriptivo del commit. Guardando el comentario como si fuera un archivo, creará el commit.

Hace el commit sin abrir el editor de texto usando el mensaje tras -m.

`git commit -m "Mensaje rápido para el commit"`

¡ATENCIÓN! Sólo se guardará lo que esté en el staging area

Comandos básicos

DESHACIENDO UN COMMIT

Para deshacer el último commit usaremos:

`git reset`

Dos estrategias:

- Deshacer el último commit pero mantener lo que había en mi working copy. Nuestro staging area queda vacío.

`git reset HEAD~1`

(o el ID del commit para retroceder más)

- Deshacer el último commit y lo que había en mi working copy de manera que todo quede como estaba antes. Nuestro staging area queda vacío.

`git reset --hard HEAD~1`

(o el ID del commit para retroceder más)

comandos básicos

VER LOS COMMITS

¿Cómo puedo ver los commits de mi repositorio?

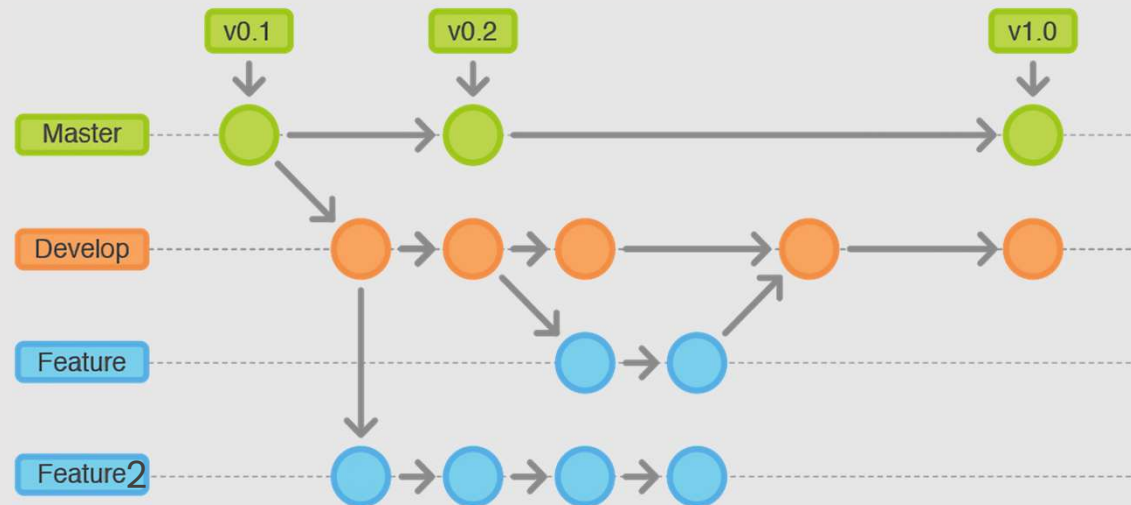
```
git log --graph --decorate --pretty=oneline  
git reflog
```



Ramas

RAMAS O BRANCHES, PARA LOS AMIGOS

- ¿Qué son?
Son directorios virtuales. Universos paralelos con un punto en común.
- ¿Para qué sirven?
Diferentes puntos de entrada al grafo.
Nos permiten desarrollar diferentes cosas en paralelo.



Ramas



¿CÓMO TRABAJAMOS CON LAS RAMAS?

- Ver las ramas existentes:

`git branch`

- Para crear una rama nueva usamos:

`git branch <NombreDeLaRama>`

¡El contenido de working copy y staging area no varía!

- Cambiar de una rama a otra:

`git checkout <NombreDeLaRama>`

¡El contenido de working copy varía!

- Eliminar una rama:

`git branch -D <NombreDeLaRama>`

¡El contenido de working copy y staging area no varía!

- No es más que eliminar un puntero del grafo (un punto de acceso al grafo)
- ¡Ojo! podemos dejar commits “inalcanzables”
- No podemos eliminar la rama en la que estamos

Ramas

MERGING: UNIENDO RAMAS

Para unir dos ramas usaremos:

`git merge <NombreDeLaRama>`

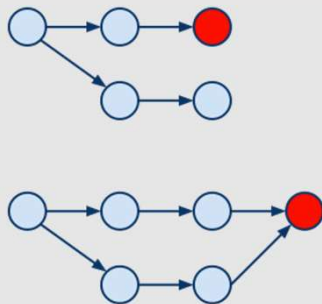
¿Quién absorbe a quién?

- La rama en la que estamos, absorbe la que le indicamos con git merge

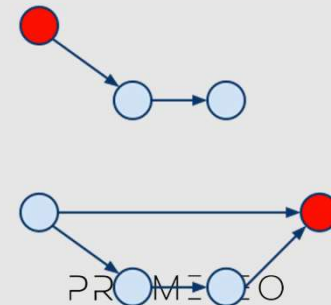
- Ejemplo:

Si estamos en 'carneAsada' y queremos hacer merge con 'salsa'
`git merge salsa`

- Cuando las ramas forman una lista. Hay dos opciones:
Con fast-forward (por defecto)



Sin fast-forward (`--no-ff`)



Ramas

RESOLVER CONFLICTOS

Cuándo hago git pull o git merge, únicamente, mi repositorio dice que hay un conflicto, es decir, da error... ¿Ahora qué hago?

1. Editar cada uno de los archivos en conflicto, quedándonos con el código que realmente nos interesa.
2. Hacer git add de esos archivos al staging-area.
3. Hacer git commit con los cambios.
4. Actualizar el repositorio remoto.

Para cancelar el merge que estábamos haciendo dejando todo como estaba.

`git merge --abort`



Trabajando en remoto

Para **CREAR UN REPOSITORIO REMOTO** podemos usar **Github**.

- Una vez creado podemos clonarlo con:
`git clone <repoUrl>`
- Es decir, copiamos un repositorio remoto para trabajar en nuestra máquina.
- Nos crea una carpeta con el mismo nombre que el repositorio.
- Ésta es la opción más cómoda
- También podemos crear un repositorio remoto usando:
`git remote add origin <UrlRepo>`
Añade <NombreRepo> como repositorio remoto



Olga M. Moreno Martín

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Owner * / Repository name *

Great repository names are short and memorable. Need inspiration? How about [jubilant-chainsaw](#)?

Description (optional)

☒ Public
Anyone on the internet can see this repository. You choose who can commit.

☐ Private
You choose who can see and commit to this repository.

Initialize this repository with:
Skip this step if you're importing an existing repository.

☒ Add a README file
This is where you can write a long description for your project. [Learn more.](#)

☒ Add .gitignore
Choose which files not to track from a list of templates. [Learn more.](#)
.gitignore template:

☒ Choose a license
A license tells others what they can and can't do with your code. [Learn more.](#)
License:

This will set `master` as the default branch. Change the default name in your [settings](#).

Grant your Marketplace apps access to this repository
You are subscribed to 1 Marketplace app

☒ LeanBoard
Whiteboard with sticky notes, connected with your GitHub issue

PROMETEO

124

Trabajando en remoto

Para **PUBLICAR NUESTROS CAMBIOS** en un repositorio remoto usaremos:

```
git push  
git push <NombreRepo> <branch>
```

Sube nuestros cambios en la rama <branch> a <NombreRepo>
Si nuestro repo no está actualizado, se rechazará

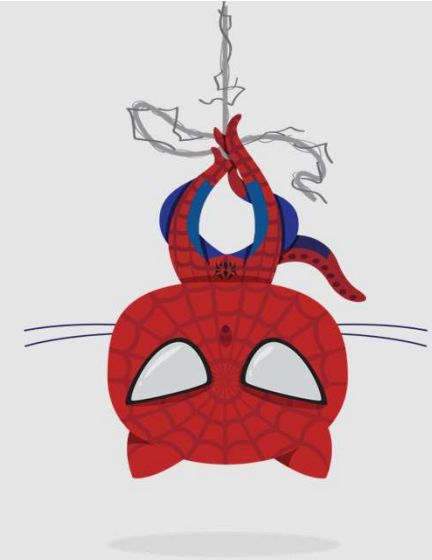
¿Cómo actualizo el repositorio?

```
git fetch  
git fetch <remote>
```

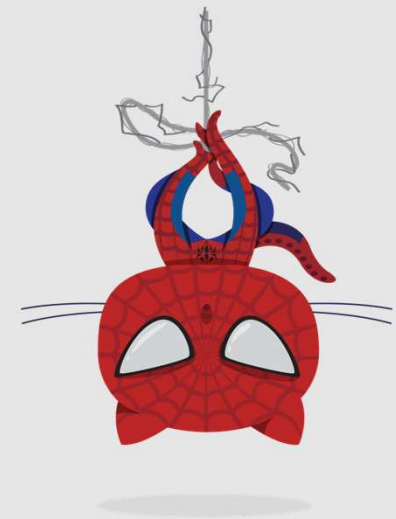
- Descarga todos los cambios de todas las ramas de <remote> desde la última vez que descargamos. Y crea nuevas ramas si es **necesario**.

```
git fetch <remote> <NombreRamaRemota>
```

- Descarga los cambios de la rama <NombreRamaRemota> de <remote> desde la última vez que descargamos. Crea una nueva rama si no la tenemos.



Trabajando en remoto



DESCARGAR Y APLICAR LOS CAMBIOS REMOTOS

`git pull`
`git pull <remote>`

`git pull` = `git fetch` + `git merge`

Perfecto para hacer actualizaciones directas

Entonces, el orden de las operaciones más corrientes queda:

- 1) `git add --all`
- 2) `git commit -m`
- 3) `git pull`
- 4) `git push`

Etiquetas: Tags y versiones

ETIQUETANDO

- Git tiene la posibilidad de etiquetar puntos específicos del historial como importantes.
- Esta funcionalidad se usa típicamente para marcar versiones de lanzamiento.
- Una etiqueta se une a un commit.

- Listar las etiquetas disponibles en Git es sencillo:

`git tag`

- Crear una etiqueta:

`git tag -a <nombreEtiqueta> -m "mensajito"`

Ejemplo:

```
$ git tag -a v1.4 -m 'my version 1.4'
```

```
$ git tag
```

```
v0.1
```

```
v1.3
```

```
v1.4
```

- La opción `-m` especifica el mensaje de la etiqueta, el cual es guardado junto con ella. Si no especificas el mensaje de una etiqueta anotada, Git abrirá el editor de texto para que lo escribas.

- Eliminar una etiqueta:

`git tag --delete <nombreEtiqueta>`



GitHub: Releases – Tags

Tag

v3.4.5

Target: main

Excellent! This tag will be created from the target when you publish this release.

Release title

WritePreview

HBI<>@↵

Previous tag: auto

Generate release notes

Describe this release

Important release

Latest

✎

🗑

This releases important things ⚡

▼ Assets2

Source code (zip)

Nov 15, 2019

Source code (tar.gz)

Nov 15, 2019

ReleasesTags

Pre-release

v1.51.0-beta0

atom-build released this 16 days ago

Compare

1.51.0-beta0

Notable Changes

- #20899 - Improve the performance of getting the list of all available atom packages

All Changes

Assets18

Latest release

v1.50.0

atom-build released this 16 days ago

Compare

1.50.0

Notable Changes

- #20804 - Fix multi cursor correct setting
- #20910 - Add async and await keywords to Rust highlighter

PROMISEO

GitHub: Issues

File Edit View Repository Branch Help

Current repository
SER421

Current branch
master

Fetch origin
Last fetched 7 minutes ago

Issues

Filters

Labels Milestones

New issue

436 Open ✓ 3,039 Closed

Author Labels Projects Milestones Assignee Sort

Clone A Repository Dialog's Statefulness Is Bad UX
#5937 opened 6 hours ago by dsanders11 ||| New Issues

can't push to repository in app (publish repository modal displayed) bug priority-2
#5936 opened 6 hours ago by FaithInMotion ||| New Issues

File selection of disabled entries possible with space help wanted priority-3
#5934 opened 13 hours ago by biolauri ||| New Issues

open in external editor visual studio code more-information-needed
#5933 opened 14 hours ago by edgale ||| New Issues

It's unclear which branch is being merged into another in the merge dialog enhancement
good first issue help wanted
#5930 opened a day ago by iAmWillShepherd 1.4.4 ||| New Issues

Add a button to the branch selector enhancement good first issue help wanted
#5929 opened a day ago by iAmWillShepherd 1.4.4 ||| New Issues

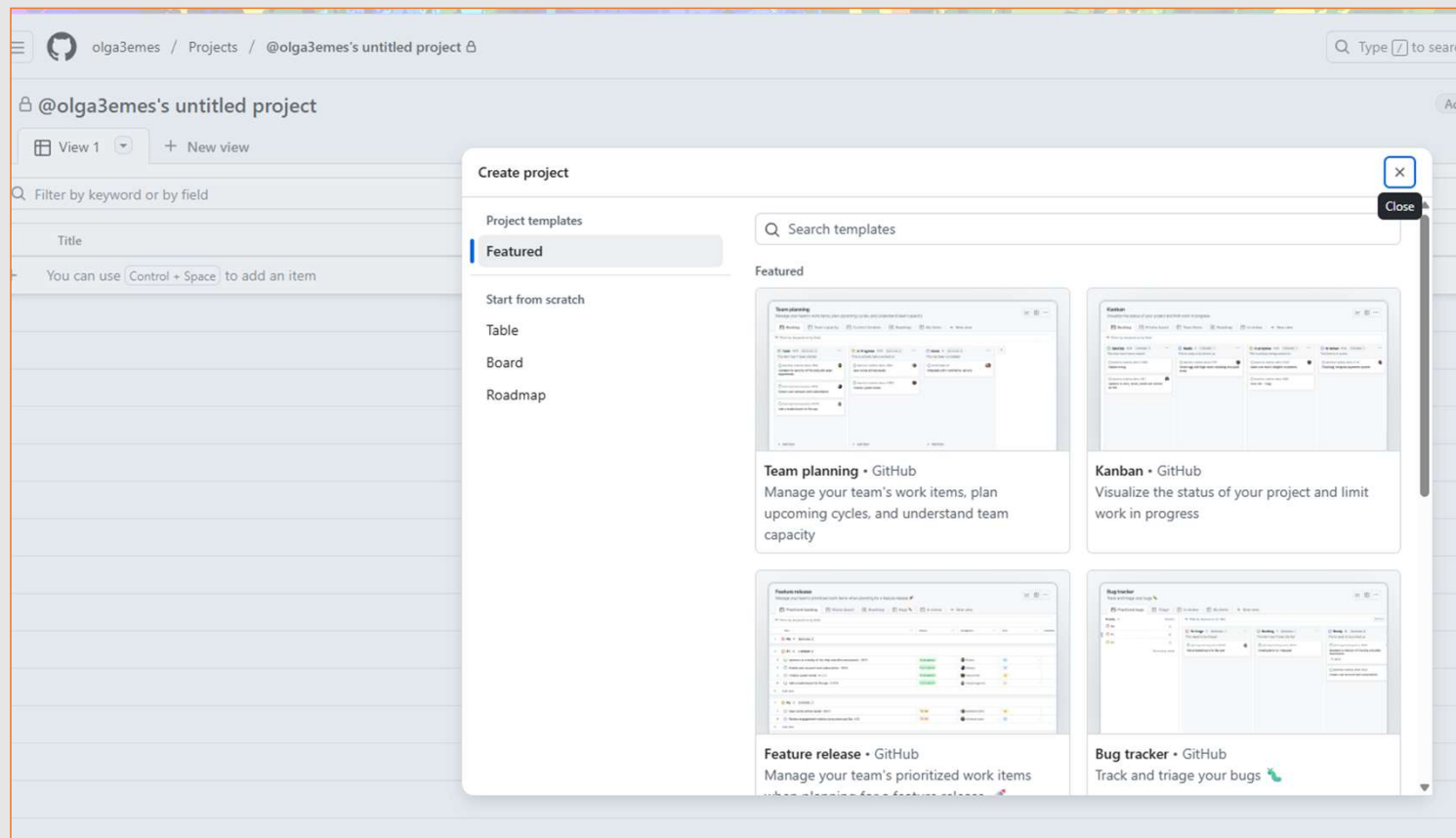
lib/git/commit functions could return SHA on success enhancement help wanted tech-debt
#5923 opened a day ago by outofambit ||| New Issues

command line more-information-needed
#5922 opened 2 days ago by wlooseven ||| New Issues

PROMETEO

GitHub : Gestor de Proyectos

- <https://learn.microsoft.com/es-es/training/paths/manage-project-lifecycle-github/>



Uso de Git y GitHub de forma profesional

Apuntes extra de cómo utilizar Git – GitHub con SSH

Configuración con clave SSH

Verifica si ya existe una clave SSH

- Primero, revisa si ya tienes generadas claves SSH para tu maquina. Abre la terminal e ingresa el siguiente comando:

`ls -al ~/.ssh`

- Si ya generaste claves SSH, deberías ver una respuesta similar a esta:

```
-rw----- 1 user_name user_name 1766 Jul 7 2018 id_rsa  
-rw-r--r-- 1 user_name user_name 414 Jul 7 2018 id_rsa.pub  
-rw----- 1 user_name user_name 12892 Feb 5 18:39 known_hosts
```

- Si tus claves ya existen, avanza a la diapositiva “Copiar tu clave SSH pública”

Configuración con clave SSH

- Si no ves ninguna respuesta o recibes un mensaje de que el archivo o directorio no existe (No such file or directory), entonces ejecuta en la terminal:

```
mkdir $HOME/.ssh
```

- Luego genera un nuevo conjunto de claves con:

```
ssh-keygen -t rsa -b 4096 -C your@email.com
```

- Ahora revisa que tus claves existan con el comando `ls -al ~/.ssh` y asegúrate de que la salida de la terminal es similar a la mostrada mas arriba.

- Las claves SSH siempre son generadas como un par de claves públicas (id_rsa.pub) y privadas (id_rsa). Es extremadamente importante que nunca reveles tu clave privada, y solo uses tu clave pública para cosas como la autenticación en GitHub.

Agrega tu clave SSH al ssh-agent

`ssh-agent` es un programa que se inicia cuando inicias sesión y almacena tus claves privadas. Para que funcione correctamente, necesita estar ejecutándose y tener una copia de tu clave privada.

- Primero, asegúrate de que `ssh-agent` esta ejecutándose con:
`eval "$(ssh-agent -s)"` # for Mac and Linux

o:

`eval `ssh-agent -s``

`ssh-agent -s` # for Windows

- Luego, añade tu clave privada al `ssh-agent` con:
`ssh-add ~/.ssh/id_rsa`

Copia tu clave SSH pública

A continuación, debes copiar tu clave SSH pública en el portapapeles:

- Para Linux o Mac, imprime el contenido de tu clave pública a la consola con:

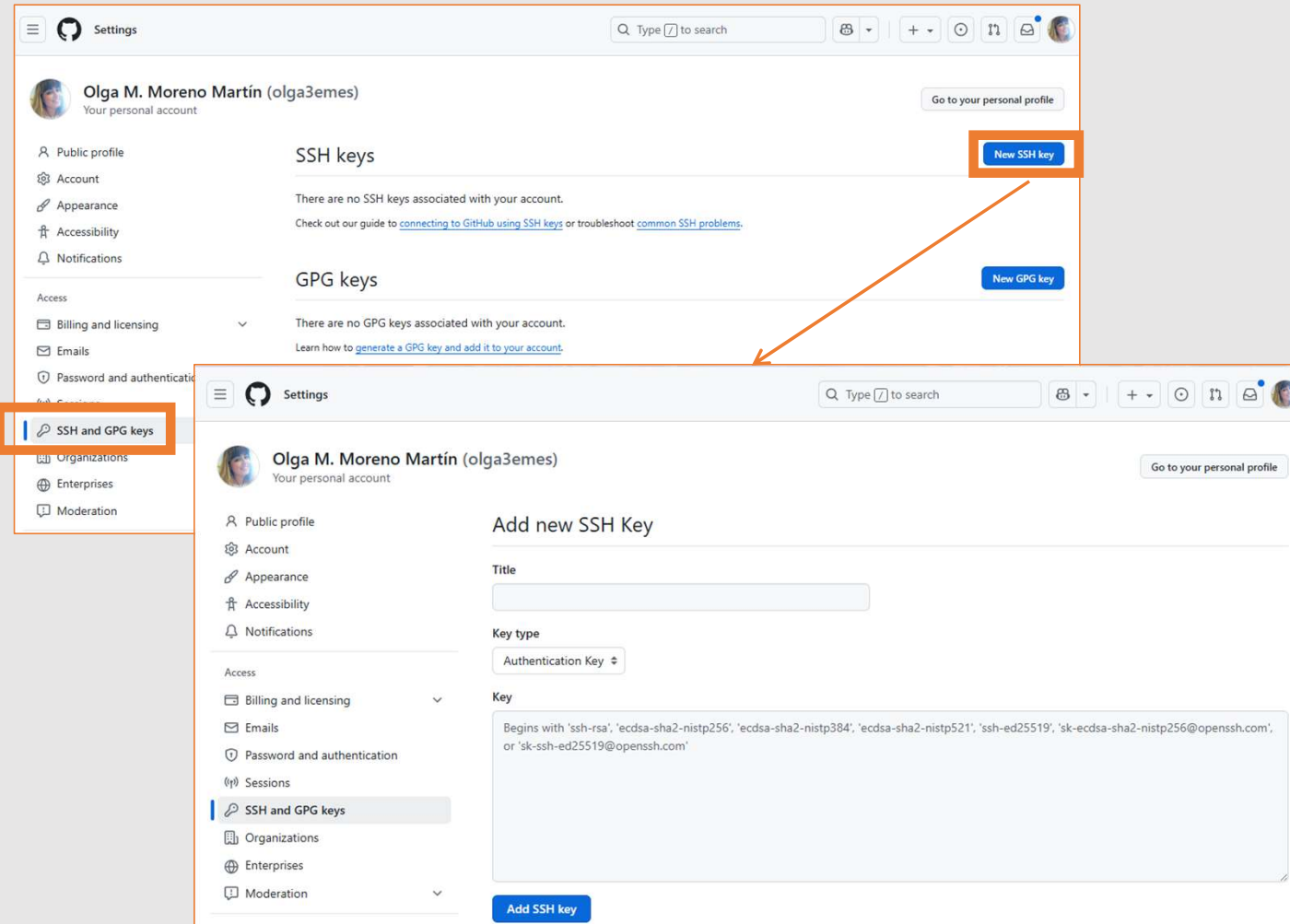
```
cat ~/.ssh/id_rsa.pub # Linux
```

- Luego selecciona y copia la respuesta.
- O para Windows simplemente ejecuta:

```
clip < ~/.ssh/id_rsa.pub # Windows
```

Añade tu clave pública SSH a GitHub

- Ve a la página de ajustes (Settings) de GitHub, click en SSH and GPG keys y en dar al botón "New SSH key":
- Luego ingresa un nombre reconocible y pega tu clave pública (id_rsa.pub) :



Comprueba la autenticación

- Finalmente, en la terminal vamos a ver si todo ha ido bien con:

```
ssh -T git@github.com
```

- Si seguiste correctamente todos los pasos, deberías ver este mensaje:

```
Hi your_user_name! You've successfully authenticated,  
but GitHub does not provide shell access.
```

¡Ya hemos terminado!

